IT4931 / IT5384 / IT4043E · SOICT · HUST

Stream Data Processing

Lecture 11 — Stream processing

Spark Structured Streaming · windows · exactly-once

Learning outcomes (CLO2, CLO3)

1. **Explain** Structured Streaming as incremental DataFrame queries
2. **Describe** triggers, output modes, event-time windows & watermarks (Spark)
3. **Outline** end-to-end exactly-once (WAL / checkpoints + replayable I/O)
4. **Preview** Apache Flink as the streaming-native peer (**L12**)

Roadmap

1. **Motivation** · DStream micro-batches (background)
2. **11.1** Spark Structured Streaming
3. **Windows / late data · FT** (Spark)
4. **Next** L12 Apache Flink internals

Motivation & DStream background

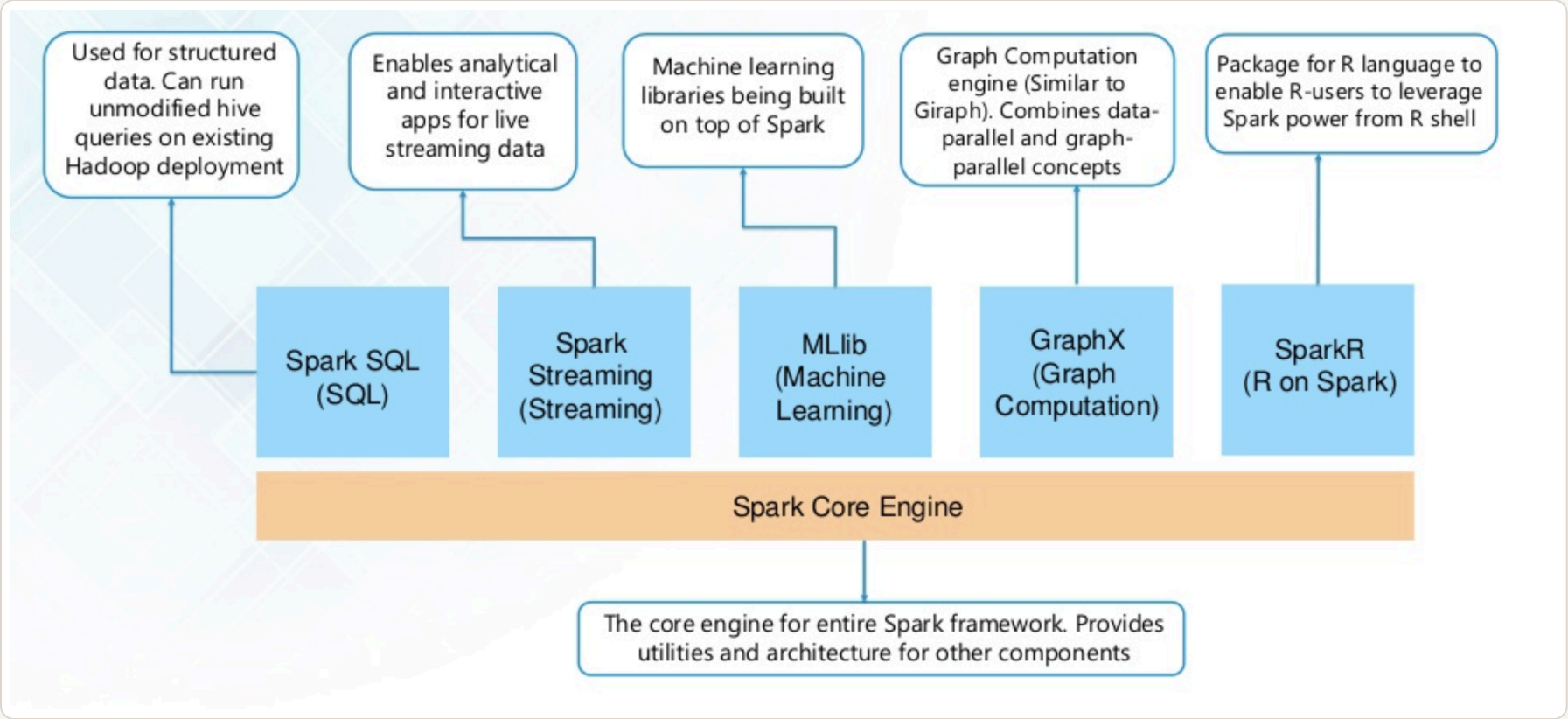
Why stream processing?

Dashboards and ops need **seconds**, not overnight batch — PPTX used a 2010 Blogger UI screenshot here.

Need	Why batch alone fails
Fresh metrics	Pageviews / sensors / clicks arrive continuously
Low latency	“Now” view must update as events land
Same analytics shape	Aggregates by time, source, geo — but **incremental**

→ Treat unbounded data as a **table that grows**; query it continuously (Structured Streaming).

Spark ecosystem (streaming in the stack)



Discretized streams (DStream) — background

Chop live data into **micro-batches** (e.g. 0.5–few seconds) → each batch is an **RDD** → same Spark engine.

Concept	Role
DStream	Sequence of RDDs
Transformations	`map` , `flatMap` , window ops, …
Output ops	`saveAs*` , `foreachRDD`

Historical model (`spark.streaming`); new apps → ***Structured Streaming****.*

DStream example — hashtags (idea)

```
# Legacy spark.streaming API (concept only)
tweets = ssc.socketTextStream(host, port)   # DStream of batches
tags = tweets.flatMap(get_tags)             # transform → new DStream
tags.foreachRDD(lambda rdd: rdd.saveAsTextFile(...))
```



Each micro-batch = one **RDD**; lineage enables recompute (next slide).

DStream fault tolerance



Replication	Lineage	Recompute
Input batches kept on multiple workers	Remember ops from the fault-tolerant input	Rebuild lost partitions elsewhere

Same idea as **batch RDDs** — streaming repeats it every micro-batch.

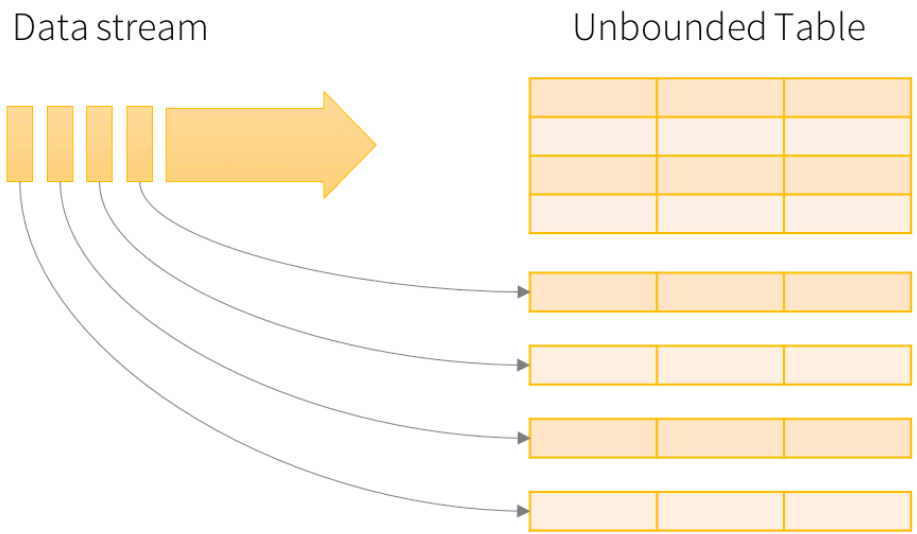
11.1 Spark Structured Streaming

Pain points with DStreams → new model

- Hard **event-time** / late data
- Awkward batch ↔ stream ↔ interactive mix
- End-to-end guarantees need careful sinks

Idea: treat the stream as an **append-only table**; run **incremental** SQL/DataFrame queries.

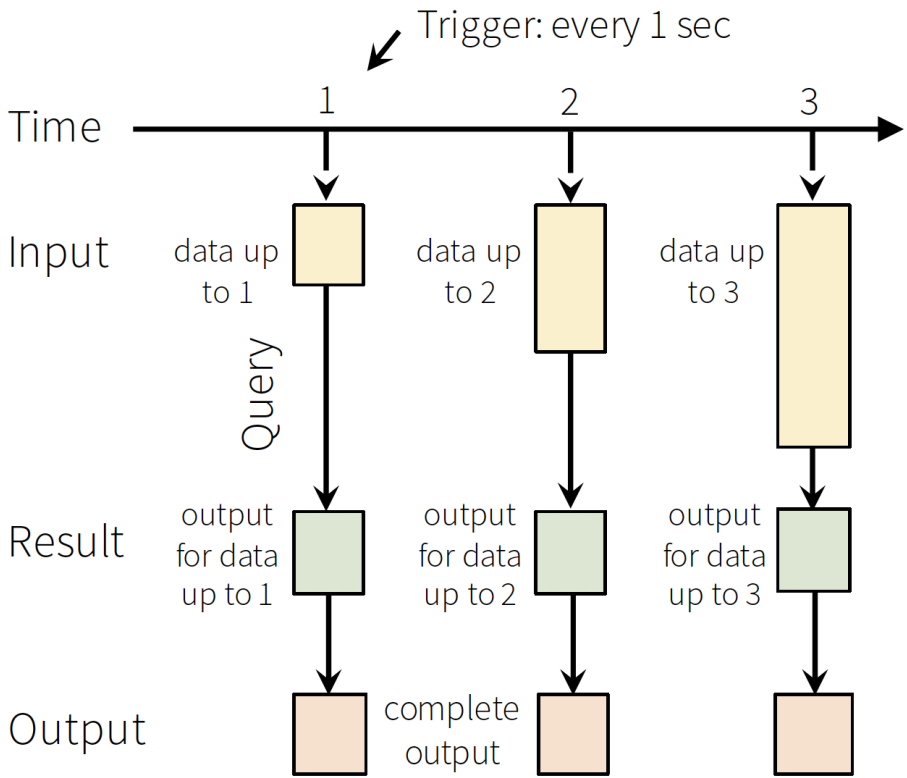
Model — input, trigger, query



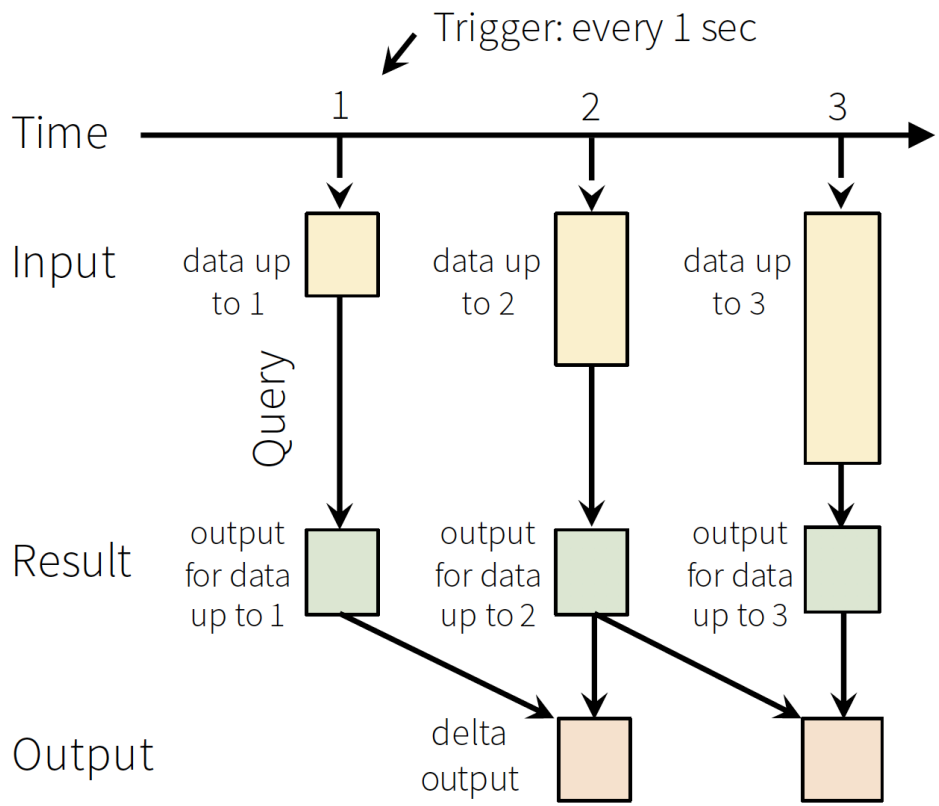
new data in the
data stream
=
new rows appended
to a unbounded table

Data stream as an unbounded table

Model — result table · Complete output



Model — Update (delta) output

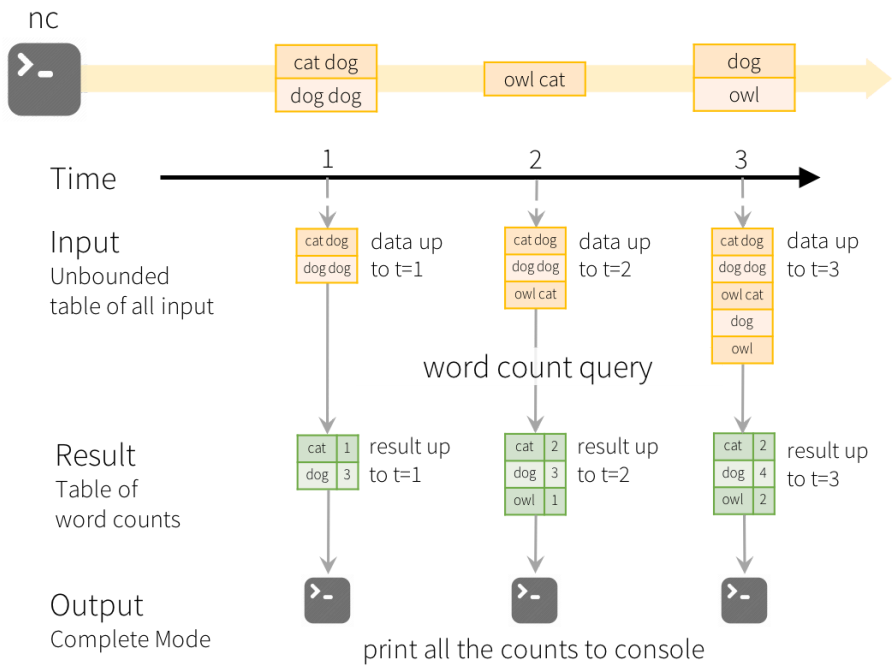


Output modes

Mode	Writes	Typical queries
Complete	Full result each trigger	Aggregations (full table)
Update	Changed rows only	Aggregations that revise keys
Append	New rows only	Filters / maps; windowed agg **after** watermark

Not every query supports every mode — Spark rejects illegal combinations at start.

Example – unbounded word count



Model of the Quick Example

Almost the same code as batch

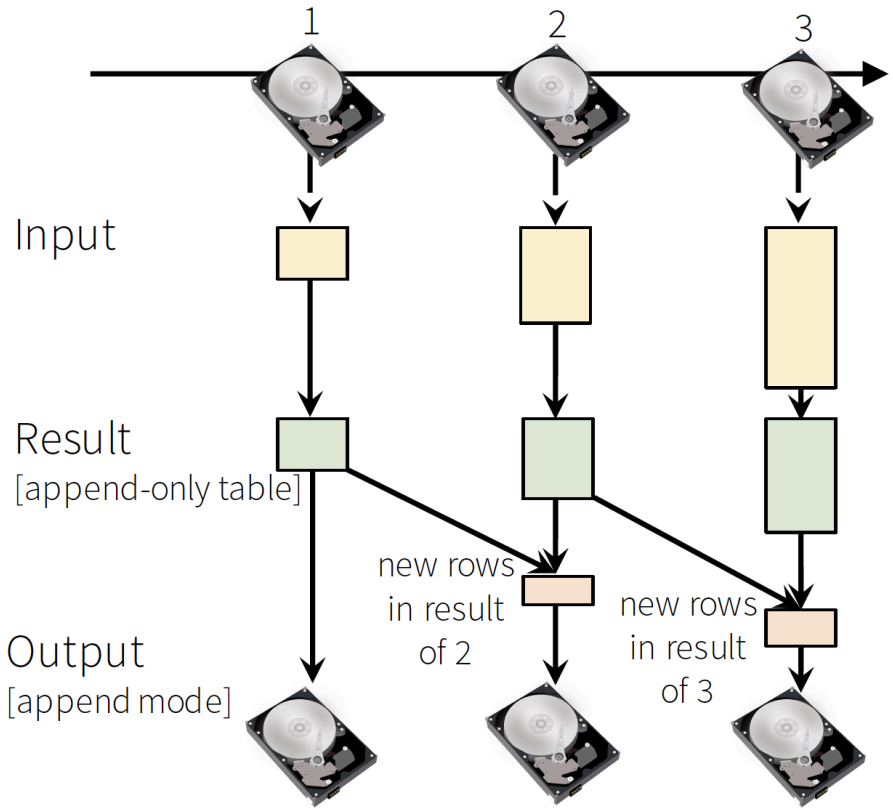
```
# Batch
df = spark.read.format("json").load(path)
# Streaming
df = spark.readStream.format("json").load(path)

out = df.select("device", "signal").where("signal > 15")

# Batch write vs streaming start
out.write.format("parquet").save(dest)
out.writeStream.format("parquet").outputMode("append").start(dest)
```

Same Catalyst engine — **incremental** physical plan.

Streaming ETL figure



Continuous aggregations

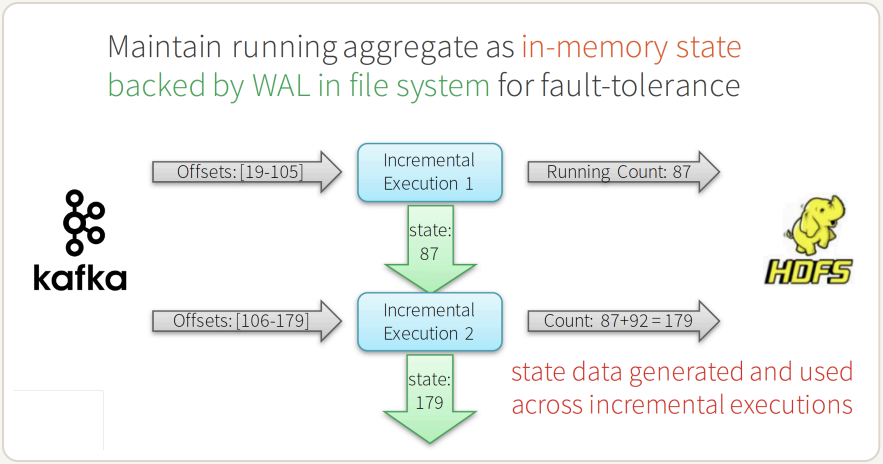
```
# Global continuous avg
df.agg(avg("signal"))

# Per-key continuous avg
df.groupBy("device_type").avg("signal")

# Event-time window (10 min)
df.groupBy("device_type", window("event_time", "10 minutes")).avg("signal")
```

Same query shape works for **batch and streaming** — event-time windows were awkward on DStreams.

State across incremental executions



Idea	Detail
State	Running aggregate kept in memory between triggers
WAL	Checkpoint offsets + state for recovery
Next batch	Starts from prior state + new offsets

Example: count so far **87**, next chunk adds **92** → result **179**.

Stream–static join (enrich)

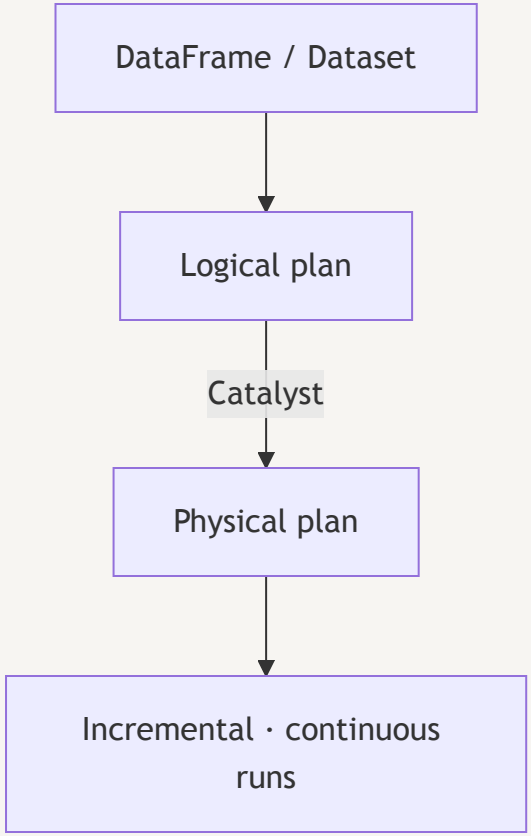
```
events = spark.readStream.format("kafka").option(...).load()
devices = spark.read.jdbc(url, "device_info")  # static

enriched = events.join(devices, "device_id")
```

Enrich streaming rows with a **dimension table** — no special "stream join" mental model for the static side.

Stream–stream joins need **watermarks + state** (careful).

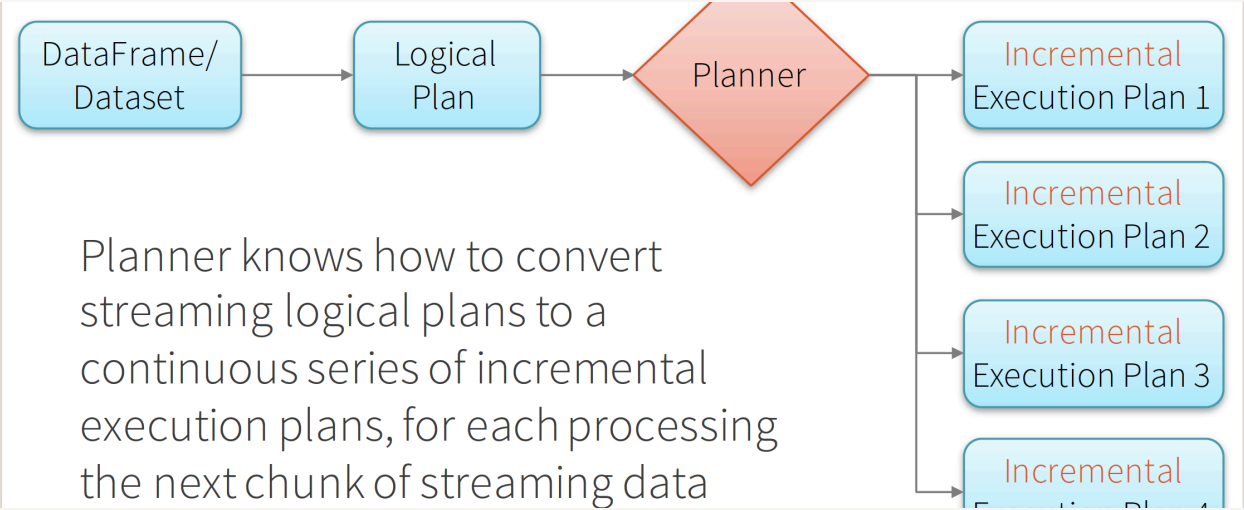
Logical vs physical execution



View	Meaning
Logical	Same table ops as batch (filter, agg, join, …)
Physical	Engine executes them **incrementally** each trigger

You write **batch-shaped** code; Spark turns it into a stream of micro-plans over new data.

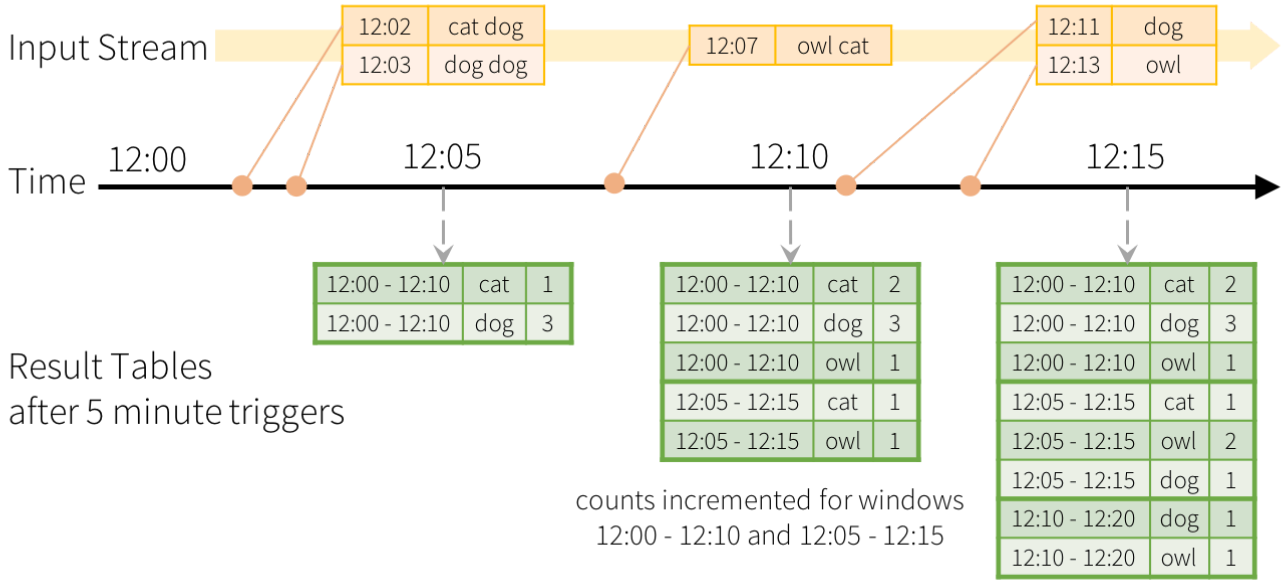
Incremental execution



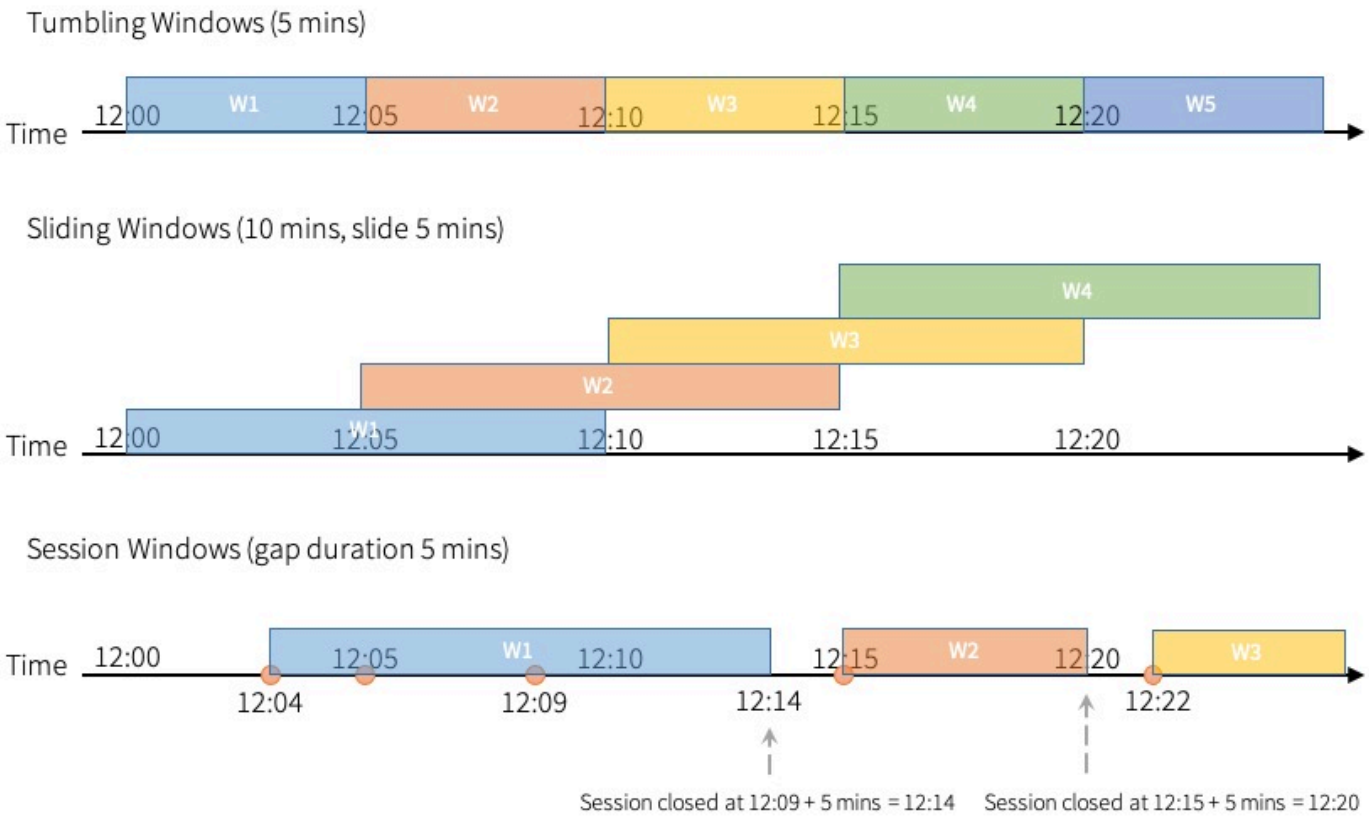
One logical streaming query → planner emits **plan 1, 2, 3, …** — each processes the next chunk of offsets.

Event time, windows, late data

Event-time window ops



Window types



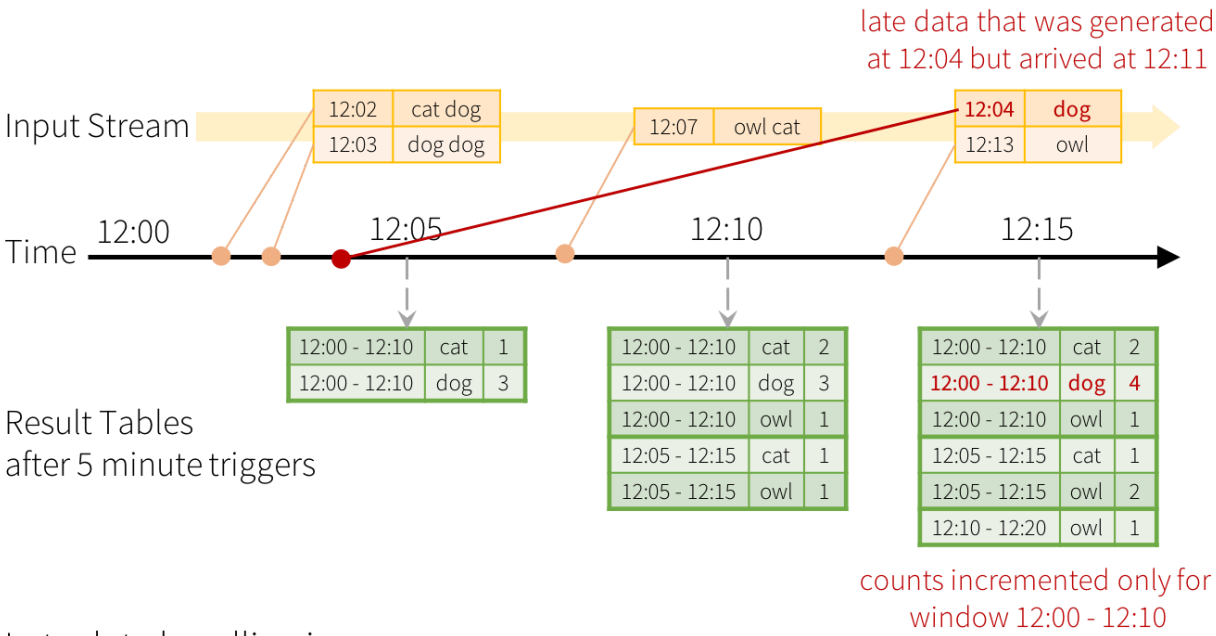
Late data & watermarking

```
words = ...  # streaming DataFrame of schema { timestamp: Timestamp, word: String }

# Group the data by window and word and compute the count of each group
windowedCounts = words \
    .withWatermark("timestamp", "10 minutes") \
    .groupBy(
        window(words.timestamp, "10 minutes", "5 minutes"),
        words.word) \
    .count()
```

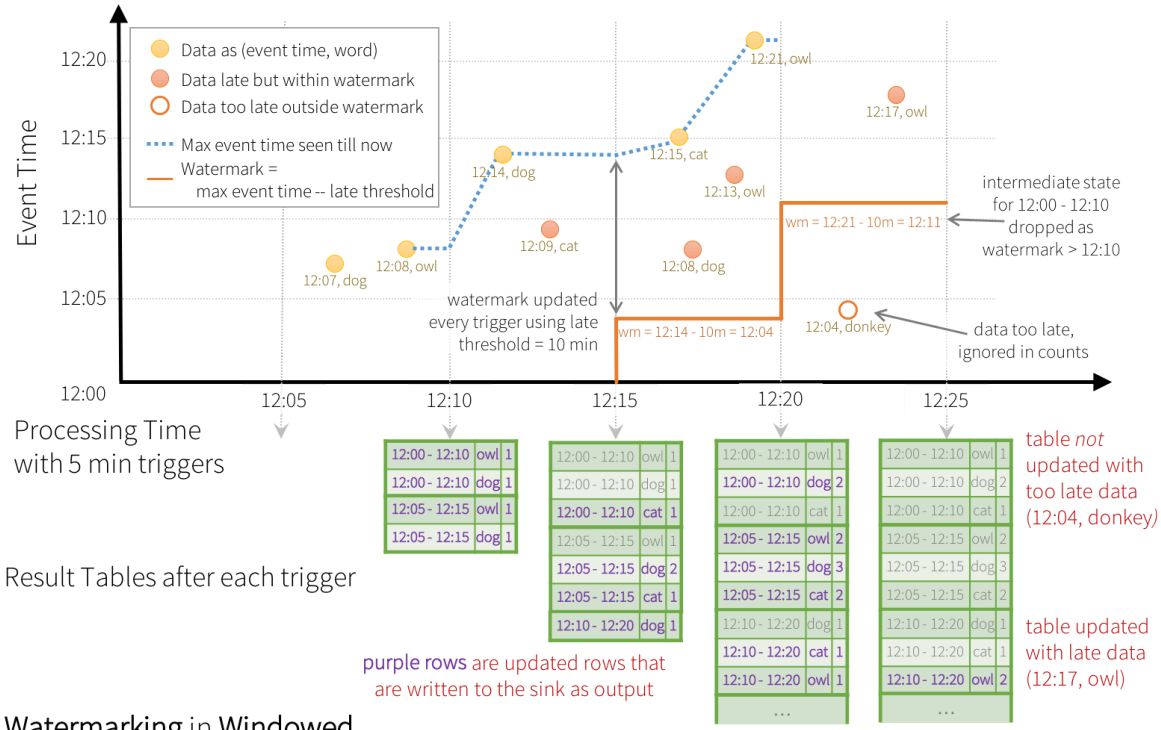
Watermark = how late data may arrive before Spark finalizes a window · `withWatermark("event_time", "10 minutes")`.

Late data walkthrough



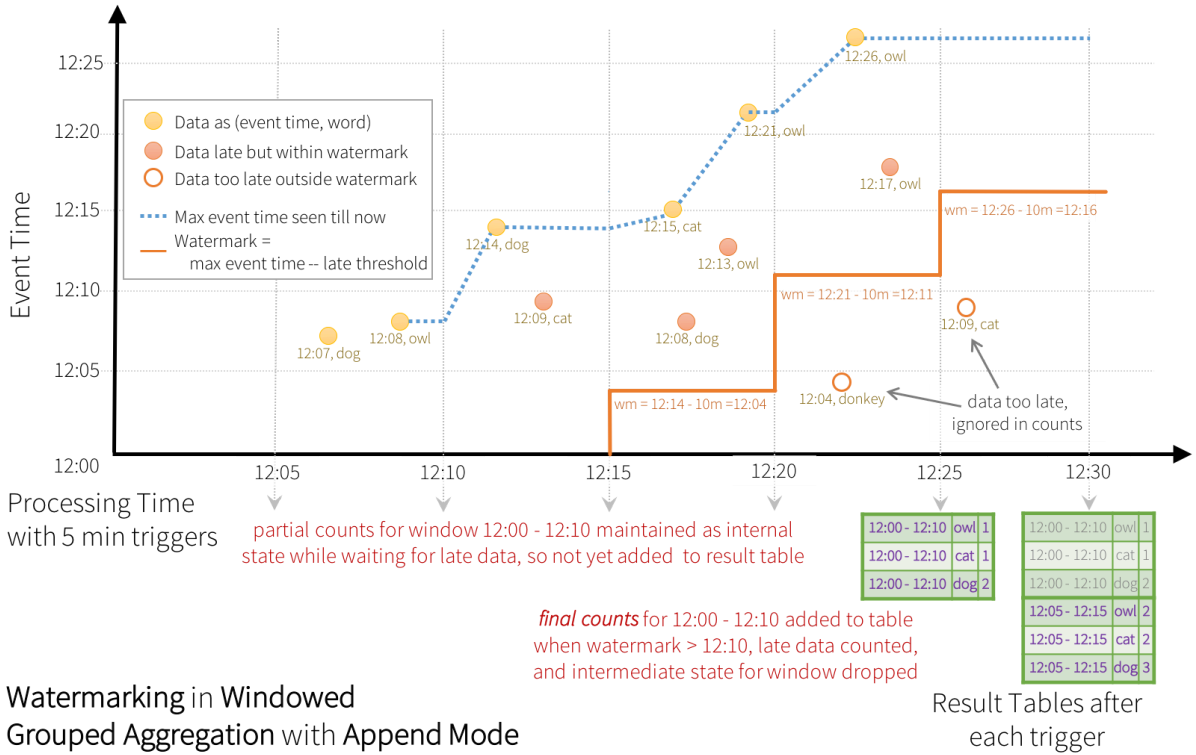
Late data handling in
Windowed Grouped Aggregation

Aggregation — Update mode + watermark



Watermarking in Windowed Grouped Aggregation with Update Mode

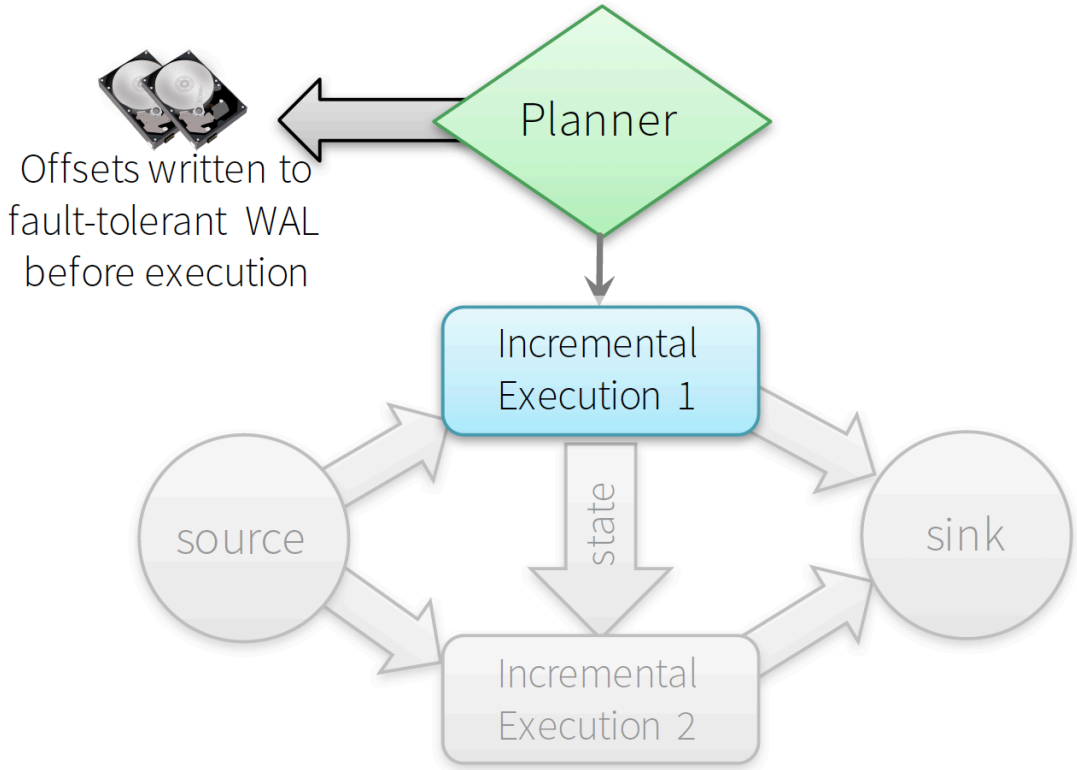
Aggregation — Append mode + watermark



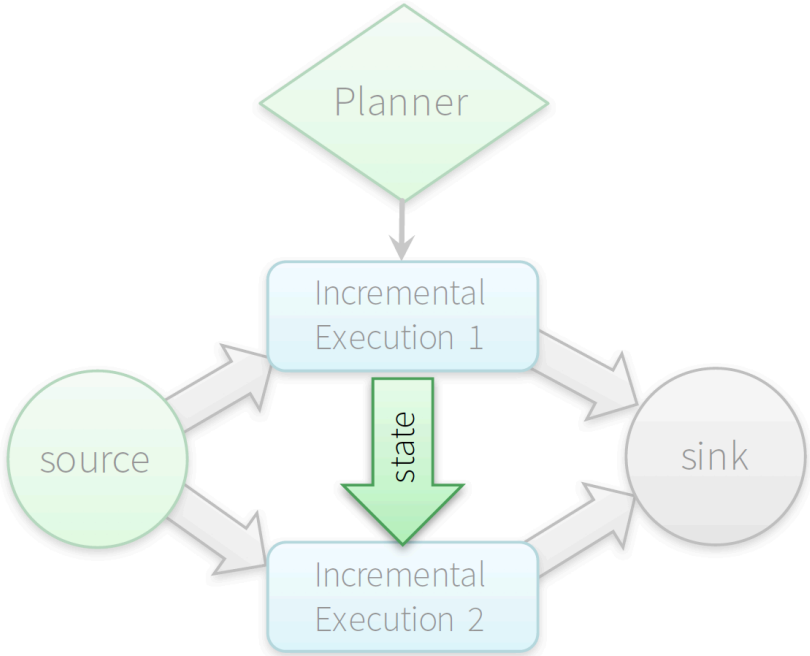
Watermarking in **Windowed Grouped Aggregation** with **Append Mode**

Exactly-once building blocks

WAL offset tracking



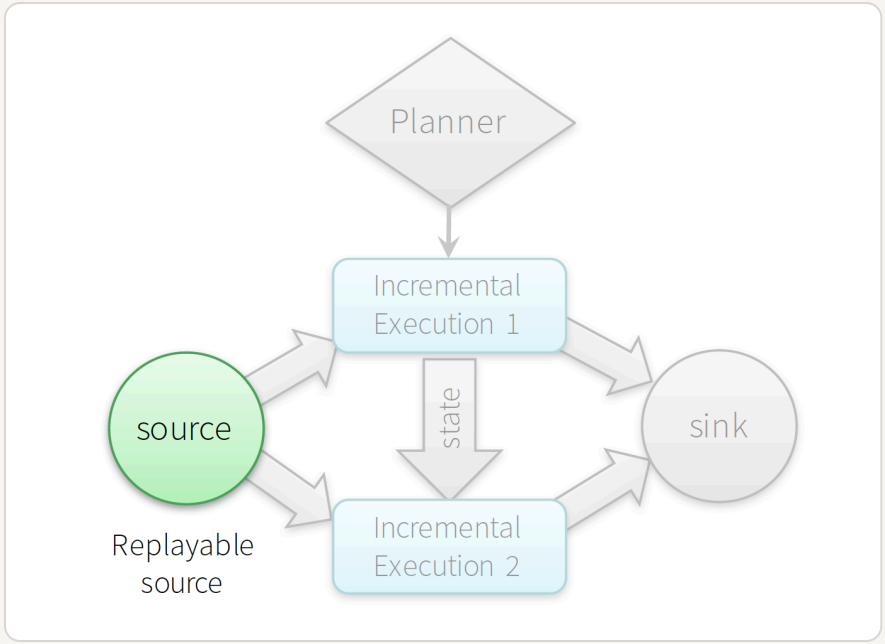
Versioned state (WAL-backed)

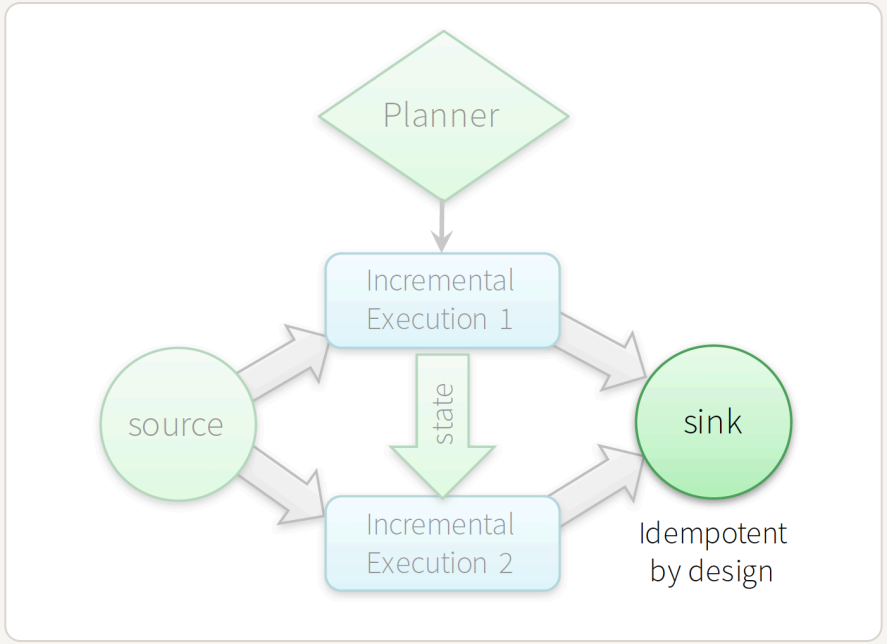


state is fault-tolerant with WAL



Replayable sources · idempotent sinks





WAL + state + sources + sinks ⇒ end-to-end exactly-once (when configured correctly).

Sources & sinks (practice)

	Examples
Sources	Kafka, files, Kinesis, rate (test)
Sinks	Kafka, Parquet/ORC files, foreachBatch, console, memory
Join	Stream–static (enrich) · stream–stream (careful state)
Query handle	`stop()` , `awaitTermination()` , status / exception

Ties to **L08 Kafka** and **L10** DataFrame engine.

Takeaways

1. Streams need **near-real-time** answers on unbounded data
2. **Spark Structured Streaming** = DataFrame queries run **incrementally**
3. **Event-time + watermark** handle late data
4. Exactly-once needs **replay + WAL + idempotent sinks**
5. Next lecture: **Apache Flink** internals (streaming-native peer)

Next

L12 — Apache Flink internals